Universidad de La Habana
Facultad de Matemática y Computación

Ingeniería de Software

Resolución de la Clase Práctica 4

Patrones de diseño orientado a objetos

Equipo: Base de Batos

Integrante	**Pregunta Asignada**
Héctor Duarte Vázquez	Problema 2: a) y b)
Alejandro Lopez Castro	Problema 2: c)
Alex Leonardo Cuervo Grillo	Problema 2: d)
Fabio Hernández Piloto	Problema 3: a) y b)
Gabriel Pérez Suárez	Problema 3: c)
Bryan Ernesto Moya Aquino	Problema 3: d) y Confeccion del documento

Fecha de entrega: 30 de octubre de 2026

Repositorio público del proyecto:
`https://github.com/el-M0ya/Base_de_Batos`

Índice

1. Problema 2: Sistema de Personalización de Bebidas **1**
1.1. Comprensión del problema . . . 1
1.2. Por qué la solución directa no sirve . . . 1
1.3. Patrón seleccionado (inciso a) . . . 2
1.4. Estructura de clases . . . 3
1.5. Implementación (inciso b) . . . 4
1.6. Descuentos por combinación (inciso c) . . . 6
1.7. Conteo de objetos y Flyweight (inciso d) . . . 7
1.8. Persistencia y reconstrucción . . . 8

2. Problema 3: Sistema de Análisis de Documentos **10**
2.1. Lectura y Comprensión del Problema . . . 10
2.2. Identificación de Conceptos del Dominio . . . 10
2.3. Reconocimiento del Problema de Diseño y Solución Ingenua . . . 11
2.4. Búsqueda, Identificación y Justificación del Patrón (Inciso a) . . . 11
2.5. Diseño de la Estructura de Clases . . . 12
2.6. Implementación de Elementos y Visitantes en C# (Inciso b) . . . 12
2.7. Integración con Composite e Iterator (Inciso c) . . . 18
2.8. Análisis Comparativo de Mantenibilidad (Inciso d) . . . 20
2.9. Restricción Técnica – Concurrencia y Paralelismo Seguro . . . 21
2.10. Prueba de Integración y Salida del Sistema . . . 23

1. Problema 2: Sistema de Personalización de Bebidas

Problema: Una cafetería quiere un sistema flexible donde los clientes puedan personalizar sus bebidas base (Café, Té, Chocolate) añadiendo múltiples complementos (Leche, Azúcar, Crema, Canela, Sirope). El costo debe calcularse dinámicamente según los complementos elegidos, y se debe poder añadir cualquier combinación de complementos.

- **a)** Identifica el patrón de diseño más adecuado, justifica tu selección basándote en las características del problema y explica la propuesta de solución considerando las clases que intervienen y las relaciones entre ellas. ¿Por qué no usar herencia múltiple o Strategy?
- **b)** Implementa con al menos 3 bebidas base y 6 complementos. Debe soportar doble porción de un complemento (ej. doble azúcar).
- **c)** Analiza: ¿Qué problema tiene el Decorator si se quieren aplicar descuentos por combinaciones (ej. Leche + Canela = 10 % off)? ¿Cómo lo resolverías?
- **d)** Mide el número de objetos creados para una bebida con 5 complementos. ¿Es aceptable en un sistema de alta concurrencia? Propón una alternativa con Flyweight si es necesario.

  Restricción: El sistema debe persistir el pedido en base de datos y reconstruir la bebida desde el registro (¿cómo se serializa un Decorator?).

1.1. Comprensión del problema

La cafetería parte de tres bebidas base —Café , Té y Chocolate – y ofrece seis complementos —Leche , Azúcar , Crema , Canela , Sirope y Caramelo — que el cliente puede combinar libremente, incluso pidiendo doble porción de uno de ellos. El precio y la descripción no están predefinidos: se construyen a medida que se agregan complementos.

El enunciado no pide crear tres bebidas, sino **componer cualquier bebida** a partir de tres bases y seis complementos. La restricción decisiva es que el resultado debe seguir siendo *una bebida* para quien la usa, aunque por dentro sea una cadena de objetos.

1.2. Por qué la solución directa no sirve

La primera idea es enumerar las combinaciones. Con N bebidas base y M complementos que pueden estar ausentes, simples o dobles, el número de clases necesarias es $N \times 3^M$. Para $N = 3$ y $M = 6$: $3 \times 3^6 = 2187$ clases.

La segunda idea es una sola clase con banderas y condicionales:

```
public decimal Costo()
{
    decimal total = precioBase;
    if (leche)    total +=   precio_leche    * porciones_leche;
    if (azucar)   total +=   precio_azucar   * porciones_azucar;
    if (crema)    total +=   precio_crema    * porciones_crema;
    if (canela)   total +=   precio_canela   * porciones_canela;
    if (sirope)   total +=   precio_sirope   * porciones_sirope;
    if (caramelo) total +=   precio_caramelo * porciones_caramelo;
    return total;
}
```

Listing 1: Solución con banderas: el problema concentrado en un método

Ambas soluciones fallan por las mismas razones:

- **Crecimiento explosivo**: cada complemento multiplica las combinaciones y obliga a crear o modificar muchas clases.

- **Violación del principio abierto/cerrado**: añadir un complemento o cambiar un precio exige tocar código existente.
- **Estados inválidos**: las banderas permiten representar pedidos incoherentes (por ejemplo, "azúcar con cero porciones").
- **Composición estática**: la combinación se decide al compilar, no cuando el cliente la pide.

Se necesita, por tanto, **añadir responsabilidades a un objeto en tiempo de ejecución, sin cambiar su clase y conservando su interfaz**.

1.3. Patrón seleccionado (inciso a)

Decorator

Decorator es el patrón adecuado porque cumple las cuatro condiciones del problema:

1. **Misma interfaz**: cada complemento implementa `IBebida`, igual que la bebida que envuelve; para el cliente, un café con leche *es* una bebida.
2. **Composición dinámica**: cada complemento envuelve a la bebida anterior y se pueden apilar tantos como se quiera, en cualquier orden.
3. **Responsabilidad añadida**: cada decorador aporta su nombre y su precio sin modificar la clase de la bebida base.
4. **Extensibilidad**: un complemento nuevo es una clase nueva; no se modifican las existentes.

Por qué no herencia múltiple

- **Explosión combinatoria**: obliga a una clase por cada subconjunto de complementos y por cada combinación de porciones (2187 en este caso).
- **Problema del diamante**: si dos complementos heredan de una clase común, aparecen ambigüedades que el lenguaje resuelve con reglas artificiales.
- **Decisión estática y acoplamiento total**: la combinación queda fijada al compilar y cambiar un precio afecta a todas las clases que lo incluyen.
- **Limitación del lenguaje**: C# no permite herencia múltiple de clases; solo de interfaces, con lo que el modelo no sería expresable tal cual.

Por qué no Strategy

Strategy y Decorator usan composición, pero responden a preguntas distintas:

- Strategy resuelve *"qué algoritmo uso"*: el contexto tiene **una** estrategia activa a la vez.
- Decorator resuelve *"qué responsabilidades acumula este objeto"*: el objeto tiene **varias** capas activas simultáneamente.

Modelar cada complemento como estrategia obligaría a una estrategia por subconjunto (de vuelta a la explosión combinatoria) o a una estrategia con banderas, que es la solución ingenua disfrazada. Strategy sí se usa en este sistema, pero para otro problema: las **políticas de descuento** (sección 1.6).

1.4. Estructura de clases

La Figura 1 muestra la solución. El cliente depende solo de `IBebida`; el decorador abstracto implementa esa misma interfaz y guarda una referencia a otra `IBebida`, lo que permite encadenar complementos.

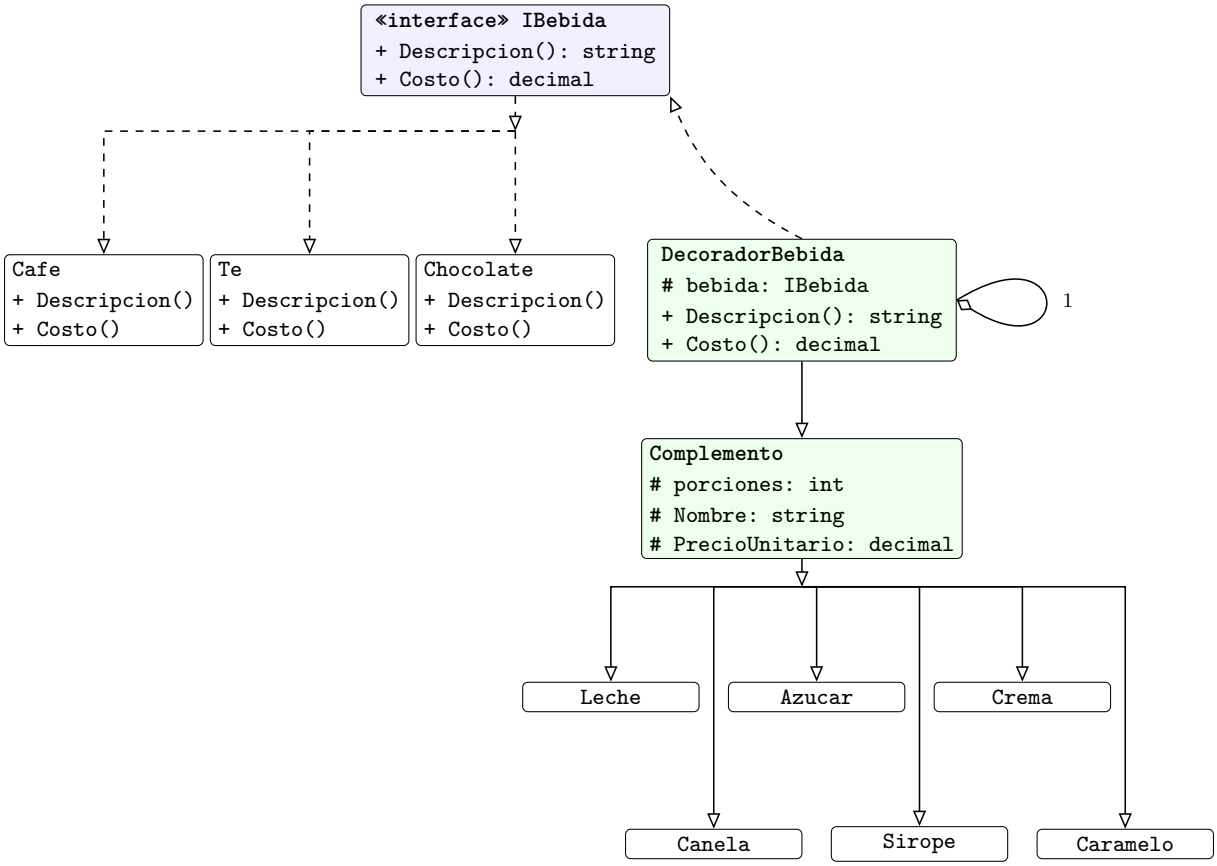


Figura 1: Diagrama de clases de la solución con Decorator.

Clase	Responsabilidad
`IBebida`	Contrato común: `Descripcion()` y `Costo()`.
`Cafe`, `Te`, `Chocolate`	Bebidas base; solo conocen su nombre y su precio.
`DecoradorBebida`	Implementa `IBebida` y envuelve otra `IBebida`; delega por defecto.
`Complemento`	Añade porciones (1 o 2), validación y precio unitario; multiplica el precio por la porción.
`Leche`, `Azucar`, `Crema`, `Canela`, `Sirope`, `Caramelo`	Cada una declara su nombre y su precio unitario.
`Pedido`	Cliente: compone la bebida y muestra descripción y total.

Tabla 1: Responsabilidades de cada clase.

1.5. Implementación (inciso b)

```
public interface IBebida
{
    string Descripcion();
    decimal Costo();
}

public abstract class DecoradorBebida : IBebida
{
    protected readonly IBebida bebida;

    protected DecoradorBebida(IBebida bebida)
        => this.bebida = bebida ?? throw new ArgumentNullException(
    nameof(bebida));

    public virtual string Descripcion() => bebida.Descripcion();
    public virtual decimal Costo() => bebida.Costo();
}
```

Listing 2: Componente abstracto y decorador abstracto

```
public abstract class Complemento : DecoradorBebida
{
    private readonly int porciones;

    protected Complemento(IBebida bebida, int porciones) : base(bebida)
    {
        if (porciones < 1 || porciones > 2)
            throw new ArgumentOutOfRangeException(nameof(porciones),
                "Solo se admite 1 o 2 porciones.");
        this.porciones = porciones;
    }

    protected abstract string Nombre { get; }
    protected abstract decimal PrecioUnitario { get; }

    public override string Descripcion()
        => base.Descripcion() + " + " + Nombre + (porciones == 2 ? " (
    doble)" : "");

    public override decimal Costo()
        => base.Costo() + PrecioUnitario * porciones;
}
```

Listing 3: Complemento con soporte de doble porción

Las tres bebidas base son clases de dos líneas:

```
public class Cafe : IBebida
{
    public string Descripcion() => "Cafe";
    public decimal Costo() => 2.00; //EJEMPLO DE COSTO
}

public class Leche : Complemento
{
    public Leche(IBebida bebida, int porciones = 1) : base(bebida,
    porciones) { }
```

```
    protected override string Nombre => "leche";
    protected override decimal PrecioUnitario => 0.50m;
}
```

Listing 4: Bebidas base y un complemento concreto de ejemplo

Bebida base	**Precio**	
Cafe	2.00	
Te	1.80	
Chocolate	2.20	
Complemento	**Precio unitario**	**Doble**
Leche	0.50	1.00
Azucar	0.20	0.40
Crema	0.60	1.20
Canela	0.30	0.60
Sirope	0.75	1.50
Caramelo	0.55	1.10

Tabla 2: Catálogo usado en la implementación (3 bebidas y 6 complementos) con precios de ejemplo.

El cliente solo manipula `IBebida`; cada asignación añade una capa:

```
IBebida bebida = new Cafe();
bebida = new Leche(bebida);
bebida = new Azucar(bebida, porciones: 2);

Console.WriteLine(bebida.Descripcion());
Console.WriteLine($"Total: {bebida.Costo():0.00}");

// Salida:
// Cafe + leche + azucar (doble)
// Total: 2.90
```

Listing 5: Composición de un pedido y salida

```
public sealed class Pedido
{
    public string Cliente { get; }
    public IBebida Bebida { get; }

    public Pedido(string cliente, IBebida bebida)
    {
        Cliente = cliente;
        Bebida = bebida;
    }

    public void Mostrar()
    {
        Console.WriteLine($"{Cliente}: {Bebida.Descripcion()}");
        Console.WriteLine($"  Total: {Bebida.Costo():0.00}");
    }
}
```

Listing 6: Cliente del patrón

1.6. Descuentos por combinación (inciso c)

El problema

Un descuento del tipo "Leche + Canela = 10 %" no encaja en la cadena de decoradores por tres motivos:

1. **Cada decorador solo ve hacia dentro**: `Leche` conoce la bebida que envuelve, pero no sabe si `Canela` está por fuera. Detectar la combinación obligaría a recorrer la cadena y acoplaría cada complemento con todos los demás.
2. **El orden altera el resultado**: aplicar un porcentaje antes o después de otro cambia el total, y la cadena fija ese orden al construirla.
3. **Los descuentos son política, no estructura**: pueden ser acumulables, exclusivos, con tope o con vigencia, y cambian con frecuencia.

Solución propuesta

Se mantiene **Decorator para el precio base** y se aplica **Strategy** para las reglas de descuento. El pedido conserva la *receta* (la lista de códigos de complementos) y las reglas se evalúan sobre esa lista, no sobre el árbol de objetos.

```
public interface IReglaDescuento
{
    string Nombre { get; }
    int Prioridad { get; }
    bool Aplica(IReadOnlyCollection<string> codigos);
    decimal Calcular(decimal subtotal);
}

public sealed class DescuentoLecheCanela : IReglaDescuento
{
    public string Nombre => "Leche + Canela (-10%)";
    public int Prioridad => 10;
    public bool Aplica(IReadOnlyCollection<string> c)
        => c.Contains("leche") && c.Contains("canela");
    public decimal Calcular(decimal subtotal) => subtotal * 0.10;
}

public static (decimal total, IReadOnlyList<string> aplicadas) Aplicar(
    decimal subtotal, IReadOnlyList<string> codigos, IEnumerable<
   IReglaDescuento> reglas)
{
    decimal descuento = 0;
    var aplicadas = new List<string>();

    foreach (IReglaDescuento r in reglas.OrderBy(r => r.Prioridad))
    {
        if (!r.Aplica(codigos)) continue;
        descuento += r.Calcular(subtotal);
        aplicadas.Add(r.Nombre);
    }
    return (subtotal - descuento, aplicadas);
}
```

Listing 7: Regla de descuento como Strategy y su aplicación

Ventajas: cada promoción es una clase independiente, la prioridad y el tope son explícitos, las reglas se prueban con una lista de códigos (sin construir objetos) y el mismo código sirve cuando

el pedido se reconstruye desde la base de datos. Como alternativa, un `DescuentoCombinacion` al final de la cadena también funciona, pero exige recorrer los decoradores y reintroduce el problema del orden.

1.7. Conteo de objetos y Flyweight (inciso d)

Una bebida con k complementos crea $k+1$ objetos: la base más k decoradores. Con 5 complementos se crean **6 objetos**. Para 100 000 pedidos concurrentes serían unos 600 000 objetos y 20–30 MB, más las cadenas temporales que genera cada llamada a `Descripcion()`. En una cafetería el volumen es bajo y el coste dominante es la preparación física, por lo que **es aceptable**. En un sistema de alta concurrencia la tasa de asignación y las pausas del recolector de basura sí se vuelven medibles, y conviene medir antes de optimizar.

Alternativa con Flyweight

Se separa el **estado intrínseco** (código, nombre y precio unitario, común a todos los pedidos) del **estado extrínseco** (qué complementos lleva el pedido y con cuántas porciones). Los objetos compartidos son inmutables y el pedido guarda solo referencias y valores pequeños.

```
public sealed class ComplementoFlyweight
{
    public string Codigo { get; }
    public string Nombre { get; }
    public decimal PrecioUnitario { get; }

    public ComplementoFlyweight(string codigo, string nombre, decimal
    precio)
    {
        Codigo = codigo;
        Nombre = nombre;
        PrecioUnitario = precio;
    }
}

public sealed class CatalogoComplementos
{
    private readonly Dictionary<string, ComplementoFlyweight> cache =
    new();

    public ComplementoFlyweight Obtener(string codigo) => cache[codigo];
    public void Registrar(ComplementoFlyweight c) => cache[c.Codigo] = c
    ;
}

public readonly record struct LineaPedido(string Codigo, int Porciones);

public static decimal CalcularCosto(
    IEnumerable<LineaPedido> lineas, CatalogoComplementos catalogo)
    => lineas.Sum(l => catalogo.Obtener(l.Codigo).PrecioUnitario * l.
    Porciones);
```

Listing 8: Flyweight del complemento y pedido basado en receta

Con Flyweight, un pedido de 5 complementos guarda 5 estructuras `LineaPedido` (12–16 bytes cada una) y **cero** decoradores; los seis `ComplementoFlyweight` son únicos en toda la aplicación. El intercambio es claro: se gana memoria y velocidad de creación, pero se pierde el polimorfismo de los decoradores. Si un complemento necesitara comportamiento propio, habría que añadir una estrategia al flyweight.

1.8. Persistencia y reconstrucción

No se serializa el árbol de objetos. Un decorador es una cadena de referencias anidadas; guardarla como objeto binario es frágil ante refactorizaciones, inseguro al deserializar tipos arbitrarios y no permite consultas. En su lugar se persiste la **receta** (bebida base + lista ordenada de complementos con sus porciones) y la cadena se reconstruye al leer.

```sql
CREATE TABLE bebida_base (
    id_bebida    SERIAL PRIMARY KEY,
    codigo       VARCHAR(20)  NOT NULL UNIQUE,
    nombre       VARCHAR(60)  NOT NULL,
    precio_base  NUMERIC(8,2) NOT NULL CHECK (precio_base >= 0)
);

CREATE TABLE complemento (
    id_complemento  SERIAL PRIMARY KEY,
    codigo          VARCHAR(20)  NOT NULL UNIQUE,
    nombre          VARCHAR(60)  NOT NULL,
    precio_unitario NUMERIC(8,2) NOT NULL CHECK (precio_unitario >= 0)
);

CREATE TABLE pedido (
    id_pedido SERIAL PRIMARY KEY,
    cliente   VARCHAR(80)  NOT NULL,
    id_bebida INTEGER      NOT NULL REFERENCES bebida_base(id_bebida),
    fecha     TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    subtotal  NUMERIC(8,2) NOT NULL CHECK (subtotal >= 0),
    descuento NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (descuento >= 0),
    total     NUMERIC(8,2) NOT NULL CHECK (total >= 0)
);

CREATE TABLE pedido_complemento (
    id_pedido       INTEGER  NOT NULL REFERENCES pedido(id_pedido) ON
   DELETE CASCADE,
    id_complemento  INTEGER  NOT NULL REFERENCES complemento(
   id_complemento),
    porciones       SMALLINT NOT NULL DEFAULT 1 CHECK (porciones BETWEEN
    1 AND 2),
    orden           SMALLINT NOT NULL CHECK (orden > 0),
    precio_unitario NUMERIC(8,2) NOT NULL CHECK (precio_unitario >= 0),
    PRIMARY KEY (id_pedido, id_complemento)
);
```

Listing 9: Esquema relacional del pedido

```
public static IBebida Reconstruir(string codigoBase,
                                  IEnumerable<LineaPedido> lineas)
{
    IBebida bebida = codigoBase switch
    {
        "cafe"      => new Cafe(),
        "te"        => new Te(),
        "chocolate" => new Chocolate(),
        _ => throw new ArgumentException($"Bebida base desconocida: {
   codigoBase}")
    };

    foreach (LineaPedido l in lineas.OrderBy(l => l.Orden))
    {
```

```
        bebida = l.Codigo switch
        {
            "leche"    => new Leche(bebida, l.Porciones),
            "azucar"   => new Azucar(bebida, l.Porciones),
            "crema"    => new Crema(bebida, l.Porciones),
            "canela"   => new Canela(bebida, l.Porciones),
            "sirope"   => new Sirope(bebida, l.Porciones),
            "caramelo" => new Caramelo(bebida, l.Porciones),
            _ => throw new ArgumentException($"Complemento desconocido: {l.Codigo}")
        };
    }

    return bebida;
}
```

Listing 10: Reconstrucción de la cadena desde la base de datos

Decisiones relevantes del esquema:

- `orden` conserva la secuencia de apilado, necesaria para la descripción y para descuentos porcentuales.
- `precio_unitario` en `pedido_complemento` es una *fotografía* del precio en el momento de la venta; si mañana sube la canela, los pedidos históricos siguen siendo correctos.
- `porciones = 2` representa la doble porción en una sola fila, sin duplicar complementos.
- El pedido y sus líneas se insertan en **una transacción**; al reconstruir se recalcula el subtotal y se compara con el guardado para detectar inconsistencias.

2. Problema 3: Sistema de Análisis de Documentos

Problema: Un software de procesamiento de documentos necesita realizar diferentes operaciones sobre elementos del documento (Párrafos, Tablas, Imágenes). Se requieren operaciones como: exportar a HTML, contar palabras, validar accesibilidad. El sistema debe ser extensible para añadir nuevas operaciones sin modificar las clases de los elementos.

- **a)** Identifica el patrón de diseño más adecuado, justifica tu selección basándote en las características del problema y explica la propuesta de solución considerando las clases que intervienen y las relaciones entre ellas. ¿Por qué no usar Strategy o polimorfismo simple?
- **b)** Implementa con 4 tipos de elementos (Párrafo, Tabla, Imagen, Lista) y 3 operaciones (ExportarHTML, ContarPalabras, ValidarAccesibilidad).
- **c)** Integra con Composite para manejar documentos jerárquicos (secciones con subsecciones) y con Iterator para recorrer elementos.
- **d)** Compara: Visitor vs. añadir métodos a cada elemento. ¿Cuándo conviene cada uno? Usa la métrica de "número de cambios por nueva operación".

  Restricción: Las operaciones deben poder ejecutarse en paralelo sobre diferentes secciones del documento (considera concurrencia y estado mutable del Visitor).

2.1. Lectura y Comprensión del Problema

El enunciado define un contexto donde coexisten dos dimensiones de variación principales:

1. **Estructura de datos (Elementos):** Clases con representaciones internas disímiles (`Párrafo`, `Tabla`, `Imagen`, `Lista`).
2. **Funcionalidad (Operaciones):** Lógicas de negocio independientes y acumulativas aplicables sobre los elementos (`ExportarHTML`, `ContarPalabras`, `ValidarAccesibilidad`).

Restricción El sistema debe permitir la incorporación continua de nuevas operaciones sin modificar las clases que representan la estructura del documento. Además, se requiere soporte jerárquico, recorrido desacoplado y ejecución paralela de operaciones sobre secciones independientes.

—

2.2. Identificación de Conceptos del Dominio

Del análisis del dominio se derivan los siguientes conceptos esenciales:

- **Elementos del Documento (Nodos):** Componentes atómicos o compuestos que conforman el contenido textual y multimedia (`Parrafo`, `Tabla`, `Imagen`, `Lista`, `Seccion`).
- **Operaciones de Análisis y Procesamiento:** Algoritmos especializados que inspeccionan o transforman la representación de cada elemento.
- **Mecanismo de Despacho:** La vía mediante la cual se vincula en tiempo de ejecución el tipo concreto del elemento con la implementación específica del algoritmo.

—

2.3. Reconocimiento del Problema de Diseño y Solución Ingenua

Si abordáramos el problema de manera ingenua mediante **polimorfismo clásico**, declararíamos cada operación directamente en una interfaz común de los elementos:

```
public interface IElementoDocumento {
    string ExportarHTML();
    int ContarPalabras();
    bool ValidarAccesibilidad();
    // Cada nueva operacion obligaria a modificar esta interfaz
}
```

Patologías detectadas:

1. **Violación del Principio Abierto/Cerrado (OCP):** Si el cliente solicita una nueva operación (por ejemplo, `ComprimirImagenes()` o `ExportarMarkdown()`), se debe modificar la interfaz base y **todas** las clases concretas que la implementan.
2. **Violación del Principio de Responsabilidad Única (SRP):** Las clases del modelo (`Parrafo`, `Tabla`) acumulan responsabilidades ajenas a su semántica de dominio (renderizado web, auditorías de accesibilidad, etc.), generando clases infladas (antipatrón *The Blob*).
3. **Polución de dependencias:** La clase `Tabla` termina dependiendo de librerías HTML o motores de análisis sintáctico.

—

2.4. Búsqueda, Identificación y Justificación del Patrón (Inciso a)

Patrón Seleccionado: Visitor (Visitante)

El patrón que resuelve con precisión este desacoplamiento es el patrón de comportamiento **Visitor**, cuya intención formal es: *«Representar una operación sobre los elementos de una estructura de objetos. Permite definir una nueva operación sin cambiar las clases de los elementos sobre los que opera»*.

Mecanismo de Doble Despacho

Dado que lenguajes orientados a objetos con ligadura única en tiempo de ejecución (como C# o Java) resuelven la sobrecarga de métodos en tiempo de compilación (*single static dispatch*), no es posible invocar dinámicamente `visitor.Visitar(elemento)` si `elemento` está referenciado bajo el tipo base `IElementoDocumento`.

El patrón Visitor sortea esta limitación mediante el mecanismo de **Doble Despacho**:

1. **Primer despacho (Dinámico):** El cliente invoca `elemento.Aceptar(visitor)`. Como la llamada es polimórfica, el entorno de ejecución resuelve la implementación concreta de `Aceptar` en el elemento real (`Parrafo`, `Tabla`, etc.).
2. **Segundo despacho (Estático dentro del contexto tipado):** Dentro del método `Aceptar`, el elemento invoca `visitor.Visitar(this)`. En este punto, la palabra clave `this` porta el tipo estático exacto de la clase concreta, direccionando el flujo a la sobrecarga correspondiente del visitante sin necesidad de operadores de inspección de tipos (`is`, `as` o *reflection*).

¿Por qué no Strategy o Polimorfismo Simple?

- **Frente a Polimorfismo Simple:** Como se demostró en el Paso 2, añadir métodos a la jerarquía de elementos acopla la representación con las operaciones, requiriendo recompilar y alterar N clases por cada operación nueva, violando OCP y SRP.
- **Frente a Strategy:** El patrón Strategy intercambia algoritmos para un **único tipo de contexto homogéneo** (o una única interfaz uniforme). En un documento, la estructura está formada por tipos con interfaces y estados completamente heterogéneos (una `Tabla` tiene filas, celdas y columnas; una `Imagen` tiene metadatos, resolución y texto alternativo; un `Parrafo` contiene texto plano o formateado). Para aplicar Strategy, el algoritmo necesitaría recibir un objeto base genérico y ejecutar verificaciones de tipos dinámicas (`if (e is Tabla)`), desembocando en el antipatrón *Spaghetti Code* y rompiendo el encapsulamiento. Visitor resuelve la heterogeneidad de tipos de forma limpia y polimórfica mediante el doble despacho.

—

2.5. Diseño de la Estructura de Clases

La arquitectura de la solución se estructura en dos jerarquías ortogonales:

1. **Jerarquía de Elementos (Visitables):**
   - Interfaz `IElementoDocumento`: Declara el método receptor `void Aceptar(IVisitorDocumento visitor)`.
   - Elementos Concretos: `Parrafo`, `Tabla`, `Imagen`, `Lista`.
2. **Jerarquía de Operaciones (Visitantes):**
   - Interfaz `IVisitorDocumento`: Declara métodos de visita específicos para cada elemento concreto: `Visitar(Parrafo p)`, `Visitar(Tabla t)`, etc.
   - Visitantes Concretos: `ExportarHtmlVisitor`, `ContarPalabrasVisitor`, `ValidarAccesibilidad`

—

2.6. Implementación de Elementos y Visitantes en C# (Inciso b)

Interfaces Base

```
using System;
using System.Collections.Generic;

public interface IVisitorDocumento {
    void Visitar(Parrafo parrafo);
    void Visitar(Tabla tabla);
    void Visitar(Imagen imagen);
    void Visitar(Lista lista);
    void Visitar(SeccionDocumento seccion);
}

public interface IElementoDocumento {
    void Aceptar(IVisitorDocumento visitor);
}
```

La interfaz `IVisitorDocumento` declara un método específico para cada tipo de elemento. También incluye `SeccionDocumento`, que se incorpora en el inciso c mediante Composite. De esta forma, tanto las hojas como los compuestos pueden recibir visitantes mediante el mismo mecanismo.

Elementos Concretos del Documento

```
public class Parrafo : IElementoDocumento {
    public string Texto { get; }

    public Parrafo(string texto) {
        Texto = texto ?? string.Empty;
    }

    public void Aceptar(IVisitorDocumento visitor) {
        if (visitor == null) throw new ArgumentNullException(nameof(
   visitor));
        visitor.Visitar(this);
    }
}

public class Tabla : IElementoDocumento {
    public int Filas => MatrizDatos.Count;
    public int Columnas =>
        Filas == 0 ? 0 : MatrizDatos[0].Count;

    public bool TieneEncabezados { get; }

    public IReadOnlyList<IReadOnlyList<string>> MatrizDatos { get; }

    public Tabla(IEnumerable<IEnumerable<string>> matrizDatos,
                 bool tieneEncabezados) {
        if (matrizDatos == null)
            throw new ArgumentNullException(nameof(matrizDatos));

        var filasCopia = new List<IReadOnlyList<string>>();
        int columnasEsperadas = -1;

        foreach (var filaOriginal in matrizDatos) {
            if (filaOriginal == null)
                throw new ArgumentException("La tabla contiene una fila
   nula.");

            var filaCopia = new List<string>();

            foreach (var celda in filaOriginal) {
                filaCopia.Add(celda ?? string.Empty);
            }

            if (columnasEsperadas == -1) {
                columnasEsperadas = filaCopia.Count;
            }
            else if (filaCopia.Count != columnasEsperadas) {
                throw new ArgumentException(
                    "Todas las filas deben tener el mismo numero de
   columnas."
                );
```

```
            }

            filasCopia.Add(filaCopia.AsReadOnly());
        }

        MatrizDatos = filasCopia.AsReadOnly();
        TieneEncabezados = tieneEncabezados;
    }

    public void Aceptar(IVisitorDocumento visitor) {
        if (visitor == null) throw new ArgumentNullException(nameof(visitor));
        visitor.Visitar(this);
    }
}

public class Imagen : IElementoDocumento {
    public string RutaFuente { get; }
    public string TextoAlternativo { get; }
    public int Ancho { get; }
    public int Alto { get; }
    public bool EsDecorativa { get; }

    public Imagen(string ruta, string alt, int ancho, int alto,
                  bool esDecorativa = false) {
        if (ancho < 0 || alto < 0)
            throw new ArgumentException("Las dimensiones no pueden ser negativas.");

        RutaFuente = ruta ?? string.Empty;
        TextoAlternativo = alt ?? string.Empty;
        Ancho = ancho;
        Alto = alto;
        EsDecorativa = esDecorativa;
    }

    public void Aceptar(IVisitorDocumento visitor) {
        if (visitor == null) throw new ArgumentNullException(nameof(visitor));
        visitor.Visitar(this);
    }
}

public class Lista : IElementoDocumento {
    public IReadOnlyList<string> Elementos { get; }
    public bool EsNumerada { get; }

    public Lista(IEnumerable<string> elementos, bool esNumerada = false)
    {
        var copia = new List<string>();

        if (elementos != null) {
            foreach (var elemento in elementos) {
                copia.Add(elemento ?? string.Empty);
            }
        }

        Elementos = copia.AsReadOnly();
```

```
        EsNumerada = esNumerada;
    }

    public void Aceptar(IVisitorDocumento visitor) {
        if (visitor == null) throw new ArgumentNullException(nameof(
    visitor));
        visitor.Visitar(this);
    }
}
```

Las propiedades de los elementos se mantienen de solo lectura. En el caso de las tablas y listas, además se realizan copias de las colecciones recibidas para evitar que una modificación externa altere sus datos durante el procesamiento.

La clase `Tabla` obtiene sus dimensiones directamente de la matriz, por lo que no existe una discrepancia entre las dimensiones declaradas y los datos almacenados. También incorpora `TieneEncabezados`, propiedad necesaria para realizar una validación de accesibilidad real.

Implementación de los Visitantes Concretos

Operación 1: Exportar a HTML

```
using System.Text;
using System.Net;

public class ExportarHtmlVisitor : IVisitorDocumento {
    private readonly StringBuilder _htmlBuilder = new StringBuilder();
    private int _nivelSeccion = 0;
    public string ObtenerResultado() => _htmlBuilder.ToString();
    private static string Codificar(string texto) {
        return WebUtility.HtmlEncode(texto ?? string.Empty);
    }
    private static string ObtenerFuenteSegura(string ruta) {
        if (string.IsNullOrWhiteSpace(ruta) || ruta.StartsWith("//"))
            return string.Empty;
        Uri uri;
        if (Uri.TryCreate(ruta, UriKind.Absolute, out uri)) {
            if (uri.Scheme != Uri.UriSchemeHttp &&
                uri.Scheme != Uri.UriSchemeHttps) {
                return string.Empty;
            }
        }
        return Codificar(ruta);
    }
    public void Visitar(Parrafo parrafo) {
        _htmlBuilder.AppendLine(
            $"<p>{Codificar(parrafo.Texto)}</p>"
        );
    }
    public void Visitar(Tabla tabla) {
        _htmlBuilder.AppendLine("<table>");
        for (int i = 0; i < tabla.MatrizDatos.Count; i++) {
            _htmlBuilder.AppendLine("<tr>");
            foreach (var celda in tabla.MatrizDatos[i]) {
                if (i == 0 && tabla.TieneEncabezados) {
                    _htmlBuilder.AppendLine(
                        $"<th scope=\"col\">{Codificar(celda)}</th>"
                    );
```

```
                }
                else {
                    _htmlBuilder.AppendLine(
                        $"<td>{Codificar(celda)}</td>"
                    );
                }
            }
            _htmlBuilder.AppendLine("</tr>");
        }
        _htmlBuilder.AppendLine("</table>");
    }
    public void Visitar(Imagen imagen) {
        string fuente = ObtenerFuenteSegura(imagen.RutaFuente);
        if (string.IsNullOrEmpty(fuente)) {
            _htmlBuilder.AppendLine(
                "<!-- Imagen omitida: fuente no permitida o vacia. -->"
            );
            return;
        }
        string alt = imagen.EsDecorativa
            ? string.Empty
            : Codificar(imagen.TextoAlternativo);
        _htmlBuilder.AppendLine(
            $"<img src=\"{fuente}\" alt=\"{alt}\" " +
            $"width=\"{imagen.Ancho}\" height=\"{imagen.Alto}\" />"
        );
    }
    public void Visitar(Lista lista) {
        string tag = lista.EsNumerada ? "ol" : "ul";
        _htmlBuilder.AppendLine($"<{tag}>");
        foreach (var item in lista.Elementos) {
            _htmlBuilder.AppendLine(
                $"<li>{Codificar(item)}</li>"
            );
        }
        _htmlBuilder.AppendLine($"</{tag}>");
    }
    public void Visitar(SeccionDocumento seccion) {
        int nivel = Math.Min(_nivelSeccion + 2, 6);
        string etiqueta = "h" + nivel;
        _htmlBuilder.AppendLine(
            $"<{etiqueta}>{Codificar(seccion.Titulo)}</{etiqueta}>"
        );
        _nivelSeccion++;
        try {
            foreach (var elemento in seccion) {
                elemento.Aceptar(this);
            }
        }
        finally {
            _nivelSeccion--;
        }
    }
}
```

El visitante genera encabezados HTML según el nivel de anidamiento de las secciones. Por ejemplo, una sección de primer nivel genera un encabezado `h2`, mientras que una subsección genera `h3`. También utiliza `th` para las celdas de encabezado de las tablas y codifica los datos

introducidos en el HTML para evitar que su contenido altere la estructura generada.

Operación 2: Contar Palabras

```
public class ContarPalabrasVisitor : IVisitorDocumento {
    public int TotalPalabras { get; private set; } = 0;

    private int Contar(string texto) {
        if (string.IsNullOrWhiteSpace(texto))
            return 0;
        return texto.Split(
            new[] { ' ', '\r', '\n', '\t' },
            StringSplitOptions.RemoveEmptyEntries
        ).Length;
    }
    public void Visitar(Parrafo parrafo) {
        TotalPalabras += Contar(parrafo.Texto);
    }
    public void Visitar(Tabla tabla) {
        foreach (var fila in tabla.MatrizDatos) {
            foreach (var celda in fila) {
                TotalPalabras += Contar(celda);
            }
        }
    }
    public void Visitar(Imagen imagen) {
        if (!imagen.EsDecorativa) {
            TotalPalabras += Contar(imagen.TextoAlternativo);
        }
    }
    public void Visitar(Lista lista) {
        foreach (var item in lista.Elementos) {
            TotalPalabras += Contar(item);
        }
    }
    public void Visitar(SeccionDocumento seccion) {
        TotalPalabras += Contar(seccion.Titulo);
        foreach (var elemento in seccion) {
            elemento.Aceptar(this);
        }
    }
}
```

El criterio de conteo incluye el texto de los párrafos, las celdas de las tablas, los elementos de las listas, el texto alternativo de las imágenes informativas y los títulos de las secciones. Las imágenes decorativas no aportan palabras al conteo.

Operación 3: Validar Accesibilidad

```
public class ValidarAccesibilidadVisitor : IVisitorDocumento {
    public List<string> ErroresAccesibilidad { get; } =
        new List<string>();
    public bool EsValido => ErroresAccesibilidad.Count == 0;
    public void Visitar(Parrafo parrafo) {
        // Un parrafo vacio no se considera automaticamente
        // un error de accesibilidad.
    }
```

```
    public void Visitar(Tabla tabla) {
        if (tabla.Filas == 0 || tabla.Columnas == 0) {
            ErroresAccesibilidad.Add(
                "La tabla no contiene datos estructurados."
            );
        }
        else if (!tabla.TieneEncabezados) {
            ErroresAccesibilidad.Add(
                "La tabla no declara una fila de encabezados."
            );
        }
    }
    public void Visitar(Imagen imagen) {
        if (!imagen.EsDecorativa &&
            string.IsNullOrWhiteSpace(imagen.TextoAlternativo)) {
            ErroresAccesibilidad.Add(
                $"La imagen '{imagen.RutaFuente}' carece de texto
    alternativo."
            );
        }
    }
    public void Visitar(Lista lista) {
        // Una lista vacia no se considera por si sola
        // un problema de accesibilidad.
    }
    public void Visitar(SeccionDocumento seccion) {
        if (string.IsNullOrWhiteSpace(seccion.Titulo)) {
            ErroresAccesibilidad.Add(
                "Se encontro una seccion sin titulo."
            );
        }
        foreach (var elemento in seccion) {
            elemento.Aceptar(this);
        }
    }
}
```

La validación se concentra en propiedades que el modelo puede comprobar: existencia de encabezados en las tablas, texto alternativo en imágenes informativas y títulos de las secciones. No se consideran automáticamente errores de accesibilidad los párrafos o las listas vacías. —

2.7. Integración con Composite e Iterator (Inciso c)

Integración Estructural con Composite

Los documentos del mundo real no son listas planas de elementos, sino estructuras jerárquicas: un documento contiene secciones, y estas pueden contener subsecciones y elementos hoja. Para representar esta organización se aplica el patrón estructural **Composite**.

La estructura queda organizada de la siguiente manera:

- **Componente común:** `IElementoDocumento`.
- **Hojas (Leaf):** `Parrafo`, `Tabla`, `Imagen` y `Lista`.
- **Compuesto (Composite):** `SeccionDocumento`.

La clase compuesta implementa la misma interfaz que los elementos hoja. Por ello, una sección puede contener tanto elementos simples como otras secciones, sin que el cliente necesite distinguir entre ambos casos.

Recorrido Desacoplado con Iterator

El patrón **Iterator** permite recorrer los elementos de una colección sin exponer directamente su representación interna. En C#, se utiliza `IEnumerable<IElementoDocumento>` junto con `IEnumerator<IElementoDocumento>` para proporcionar un recorrido secuencial.

La clase también incorpora un recorrido en profundidad mediante `yield return`, que permite obtener todos los nodos de la jerarquía de forma perezosa.

```
using System.Collections;
using System.Collections.Generic;

public class SeccionDocumento :
    IElementoDocumento, IEnumerable<IElementoDocumento> {
    public string Titulo { get; }
    private readonly List<IElementoDocumento> _elementos =
        new List<IElementoDocumento>();
    public SeccionDocumento(string titulo) {
        Titulo = titulo ?? string.Empty;
    }
    public void Agregar(IElementoDocumento elemento) {
        if (elemento == null)
            throw new ArgumentNullException(nameof(elemento));
        _elementos.Add(elemento);
    }
    public bool Eliminar(IElementoDocumento elemento) {
        return _elementos.Remove(elemento);
    }
    // La seccion participa directamente en Visitor.
    public void Aceptar(IVisitorDocumento visitor) {
        if (visitor == null)
            throw new ArgumentNullException(nameof(visitor));
        visitor.Visitar(this);
    }
    // Iterator: recorrido de los elementos directos.
    public IEnumerator<IElementoDocumento> GetEnumerator() {
        return _elementos.GetEnumerator();
    }
    IEnumerator IEnumerable.GetEnumerator() {
        return GetEnumerator();
    }
    // Recorrido en profundidad de toda la jerarquia.
    public IEnumerable<IElementoDocumento> RecorrerEnProfundidad() {
        foreach (var elemento in _elementos) {
            yield return elemento;
            if (elemento is SeccionDocumento subSeccion) {
                foreach (var nodo in subSeccion.RecorrerEnProfundidad())
    {
                    yield return nodo;
                }
            }
        }
    }
}
```

Colaboración entre Composite y Visitor

La colaboración entre ambos patrones se produce cuando una sección recibe un visitante y lo acepta mediante `Visitar(SeccionDocumento)`. Cada visitante procesa el título de la sección

y posteriormente recorre sus elementos, invocando `Aceptar()` sobre cada uno de ellos.

De esta forma, el recorrido continúa recursivamente cuando se encuentra otra sección y se utiliza el método especializado correspondiente cuando se encuentra un elemento hoja.

Esta decisión permite que las operaciones conserven la jerarquía del documento: el exportador genera encabezados según la profundidad de las secciones, el contador incluye sus títulos y el validador puede detectar secciones sin título.

Condición para el Procesamiento Concurrente

La clase permite agregar y eliminar elementos durante la construcción del documento. Sin embargo, una vez que comienza el procesamiento concurrente, la estructura debe permanecer estable: ningún hilo debe modificar las colecciones mientras otros las recorren.

Además, las secciones que se procesan en paralelo deben corresponder a subárboles independientes, sin elementos compartidos entre ellos. Esto evita recorrer dos veces un mismo elemento y elimina conflictos derivados de modificaciones simultáneas. —

2.8. Análisis Comparativo de Mantenibilidad (Inciso d)

Evaluación Formal de Métricas de Mantenimiento

Para evaluar objetivamente la extensibilidad, comparamos el patrón Visitor contra el diseño tradicional de añadir métodos a cada clase:

Métrica de Mantenimiento	Métodos en Cada Elemento	Patrón Visitor
Cambios al añadir **1 nueva operación**	$N + 1$ clases	**1 clase nueva** (*Visitor*)
Cambios al añadir **1 nuevo elemento**	**1 clase nueva**	$M + 2$ modificaciones
Acoplamiento con dependencias externas	Alto en el modelo de dominio	Encapsulado en el Visitor
Riesgo de regresión en el modelo	Alto (se tocan todas las entidades)	Nulo (el modelo queda intact

Tabla 3: Comparativa dimensional entre diseño tradicional y Visitor (N: cantidad de elementos, M: cantidad de operaciones).

Criterios de Elección (¿Cuándo conviene cada uno?)

1. **Conviene usar Visitor cuando:**
   - La jerarquía de clases de elementos es **estable y cambia con poca frecuencia**, pero el número de operaciones a realizar sobre ellos es **abierto, dinámico y en crecimiento**.
   - Se requiere realizar operaciones que no corresponden a la responsabilidad nuclear de los objetos del dominio, evitando la polución de código y garantizando SRP.
   - Se deben acumular estados a lo largo del recorrido (como sumatorias o conteos globales).
2. **Conviene añadir métodos directamente a cada elemento cuando:**
   - La jerarquía de clases cambia constantemente (se añaden nuevos tipos de elementos frecuentemente), lo que en Visitor obligaría a modificar la interfaz `IVisitor` y todos los visitantes ya construidos.
   - El conjunto de operaciones es reducido, cerrado e íntimamente ligado a la representación interna privada del elemento.

—

2.9. Restricción Técnica – Concurrencia y Paralelismo Seguro

Diagnóstico del Problema de Concurrencia

Los visitantes concretos mantienen estado mutable durante su ejecución. Por ejemplo, `ContarPalabrasVisitor` modifica el acumulador `TotalPalabras`, mientras que `ValidarAccesibilidadVisitor` modifica la colección `ErroresAccesibilidad`.

Si varios hilos comparten una misma instancia de estos visitantes, pueden producirse **condiciones de carrera (Race Conditions)**: dos ejecuciones podrían modificar simultáneamente el mismo estado y generar resultados incorrectos.

El problema no se encuentra en el patrón Visitor en sí, sino en compartir entre hilos una instancia que mantiene datos mutables sin sincronización.

Estrategia de Solución: Visitantes Independientes y Reducción

La solución consiste en procesar secciones independientes y crear una instancia local del visitante para cada sección. Cada ejecución almacena su resultado en una posición diferente de un arreglo de resultados.

El procedimiento es el siguiente:

1. Dividir el documento en secciones cuyos subárboles sean independientes.
2. Ejecutar las operaciones mediante `Parallel.For`.
3. Crear un visitante nuevo en cada iteración, evitando compartir acumuladores, listas o constructores HTML.
4. Al finalizar las operaciones, reunir los resultados parciales en el orden correspondiente a las secciones originales.

Implementación del Procesamiento Paralelo

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Net;
using System.Threading.Tasks;

public class ProcesadorDocumentosParalelo {

    // Exportacion HTML paralela.
    public static string ExportarHtmlParalelo(
        string tituloDocumento,
        IReadOnlyList<SeccionDocumento> secciones) {
        if (secciones == null)
            throw new ArgumentNullException(nameof(secciones));
        string[] fragmentos = new string[secciones.Count];
        Parallel.For(0, secciones.Count, i => {
            // Cada iteracion posee su propio visitante.
            var visitanteLocal = new ExportarHtmlVisitor();
            secciones[i].Aceptar(visitanteLocal);

            // Cada iteracion escribe en una posicion independiente.
            fragmentos[i] = visitanteLocal.ObtenerResultado();
        });
        var resultado = new StringBuilder();
        resultado.AppendLine(
```

```
            $"<h1>{WebUtility.HtmlEncode(tituloDocumento ?? string.Empty
    )}</h1>"
        );
        // Se conserva el orden original de las secciones.
        foreach (var fragmento in fragmentos) {
            resultado.AppendLine(fragmento);
        }
        return resultado.ToString();
    }

    // Conteo de palabras paralelo.
    public static int ContarPalabrasParalelo(
        string tituloDocumento,
        IReadOnlyList<SeccionDocumento> secciones) {
        if (secciones == null)
            throw new ArgumentNullException(nameof(secciones));
        int[] resultadosParciales = new int[secciones.Count];
        Parallel.For(0, secciones.Count, i => {
            var visitanteLocal = new ContarPalabrasVisitor();
            secciones[i].Aceptar(visitanteLocal);
            resultadosParciales[i] = visitanteLocal.TotalPalabras;
        });
        // Se incluye el titulo del documento y se reducen
        // los resultados obtenidos en cada seccion.
        int total = ContarTexto(tituloDocumento);
        foreach (var parcial in resultadosParciales) {
            total += parcial;
        }
        return total;
    }

    // Validacion de accesibilidad paralela.
    public static List<string> ValidarAccesibilidadParalelo(
        string tituloDocumento,
        IReadOnlyList<SeccionDocumento> secciones) {
        if (secciones == null)
            throw new ArgumentNullException(nameof(secciones));
        var erroresPorSeccion = new List<string>[secciones.Count];
        Parallel.For(0, secciones.Count, i => {
            var visitanteLocal = new ValidarAccesibilidadVisitor();
            secciones[i].Aceptar(visitanteLocal);
            erroresPorSeccion[i] = visitanteLocal.ErroresAccesibilidad;
        });
        var erroresGlobales = new List<string>();
        if (string.IsNullOrWhiteSpace(tituloDocumento)) {
            erroresGlobales.Add(
                "El documento principal carece de titulo."
            );
        }
        // Fusion de resultados respetando el orden de las secciones.
        for (int i = 0; i < secciones.Count; i++) {
            foreach (var error in erroresPorSeccion[i]) {
                erroresGlobales.Add(
                    $"[{secciones[i].Titulo}] {error}"
                );
            }
        }
        return erroresGlobales;
```

```
    }
    private static int ContarTexto(string texto) {
        if (string.IsNullOrWhiteSpace(texto))
            return 0;

        return texto.Split(
            new[] { ' ', '\r', '\n', '\t' },
            StringSplitOptions.RemoveEmptyEntries
        ).Length;
    }
}
```

Justificación de la Seguridad Concurrente

La implementación evita compartir instancias de visitantes entre ejecuciones paralelas. Cada iteración utiliza sus propios acumuladores, listas de errores y constructores HTML.

Los arreglos de resultados también evitan conflictos: cada iteración escribe exclusivamente en la posición que le corresponde. La fusión se realiza una vez finalizado el procesamiento paralelo.

Es importante precisar que se crea **una instancia de visitante por sección procesada**, no necesariamente una por hilo. El planificador de tareas de .NET puede ejecutar varias iteraciones utilizando el mismo hilo, pero cada iteración conserva su propio visitante.

La seguridad de esta estrategia depende de dos condiciones:

- Los subárboles asignados a diferentes iteraciones no deben compartir elementos que puedan producir resultados duplicados.
- La estructura del documento no debe modificarse mientras se ejecutan las operaciones paralelas.

Por tanto, la solución combina aislamiento del estado mutable, procesamiento paralelo de secciones independientes y una fase final de reducción de resultados.

2.10. Prueba de Integración y Salida del Sistema

El siguiente programa consolida la solución integrando **Visitor**, **Composite**, **Iterator** y el procesamiento concurrente:

```
public class Programa {
    public static void Main() {

        // 1. Construccion de la jerarquia (Composite).
        var documento = new SeccionDocumento("Documento Maestro");
        var seccion1 = new SeccionDocumento("Seccion Introductoria");
        seccion1.Agregar(new Parrafo("Analisis y Diseno de Software
    Orientado a Objetos.") );
        seccion1.Agregar(
            new Imagen("diagrama.png"  ,  "Diagrama de clases UML",
                       800 , 600 ) );
        // Se crea una subseccion real.
        var subseccion = new SeccionDocumento(
            "Conceptos Fundamentales"
        );
        subseccion.Agregar(
            new Parrafo(
                "Visitor permite separar las operaciones de los
    elementos."
```

```
            )
        );
        seccion1.Agregar(subseccion);
        var seccion2 = new SeccionDocumento(
            "Seccion de Metricas"
        );
        var datos = new List<List<string>> {
            new List<string> { "Patron", "Tipo" },
            new List<string> { "Visitor", "Comportamiento" },
            new List<string> { "Composite", "Estructural" }
        };
        // La primera fila contiene los encabezados.
        seccion2.Agregar(
            new Tabla(datos, tieneEncabezados: true)
        );
        seccion2.Agregar(
            new Lista(
                new List<string> {
                    "Alta Cohesion",
                    "Bajo Acoplamiento"
                }
            )
        );
        documento.Agregar(seccion1);
        documento.Agregar(seccion2);
        // 2. Recorrido de la jerarquia mediante Iterator.
        Console.WriteLine("=== RECORRIDO DEL DOCUMENTO ===");
        foreach (var nodo in documento.RecorrerEnProfundidad()) {
            Console.WriteLine(nodo.GetType().Name);
        }
        // 3. Preparacion de las secciones independientes.
        var secciones = new List<SeccionDocumento> {
            seccion1,
            seccion2
        };
        // 4. Ejecucion paralela de las tres operaciones.
        string html =
            ProcesadorDocumentosParalelo.ExportarHtmlParalelo(
                documento.Titulo,
                secciones
            );
        int totalPalabras =ProcesadorDocumentosParalelo.
    ContarPalabrasParalelo(
                documento.Titulo,
                secciones );
        var errores = ProcesadorDocumentosParalelo.
    ValidarAccesibilidadParalelo(
                documento.Titulo,
                secciones);
        // 5. Presentacion de resultados.
        Console.WriteLine("=== REPORTE HTML GENERADO ===");
        Console.WriteLine(html);
        Console.WriteLine("=== METRICAS Y ANALISIS CONCURRENTE ===");
        Console.WriteLine(
            $"Total de palabras contabilizadas: {totalPalabras}");
        Console.WriteLine(
            $"Errores de accesibilidad encontrados: {errores.Count}" );
        foreach (var error in errores) {
```

```
74                 Console.WriteLine(error);
75             }
76         }
77 }
```

—

Conclusiones Metodológicas

1. **Modularidad y Principio OCP:** Mediante el patrón **Visitor**, incorporar operaciones analíticas futuras (como validación ortográfica o exportación a Latex o PDF) se logra creando una clase que implementa `IVisitorDocumento`, manteniendo el código existente intacto y testeado.

2. **Sinergia de Patrones:** La combinación de **Composite** (para la estructura recursiva), **Iterator** (para el recorrido abstracto) y **Visitor** (para la ejecución desacoplada de operaciones) representa un estándar de excelencia en la arquitectura de procesadores de estructuras complejas de documentos.

3. **Concurrencia limpia:** Al diseñar el patrón con confinamiento de estado por hilo (*Thread Confinement*) durante la fase de mapeo, se satisfacen plenamente las restricciones de rendimiento y sincronización en entornos de alta concurrencia.